EXAMEN PARCIAL: SISTEMAS EMBEBIDOS

Ingeniería en Mecatrónica – 4.° Año

Cátedra: Sistemas Embebidos
Tema: Sistemas Operativos de Tiempo Real (FreeRTOS en ESP32), Periféricos, Comunicación Serie (Protocolo UNER) y Stack TCP/IP (Wi-Fi)
Tiempo Estimado: 3 a 4 horas (Laboratorio con SIMULADOR SCARA 2DOF en Qt)
Entorno de Desarrollo: ESP-IDF en ESP-IDE / C

1. Contexto del Sistema

Se requiere desarrollar el firmware de control para un brazo robótico **SCARA de 2 Grados de Libertad (2DOF)** equipado con un elevador de lápiz (**PEN**). El firmware se ejecutará sobre un microcontrolador **ESP32** e interactuará con un entorno de simulación bidireccional desarrollado en **Qt** (Figura 1).

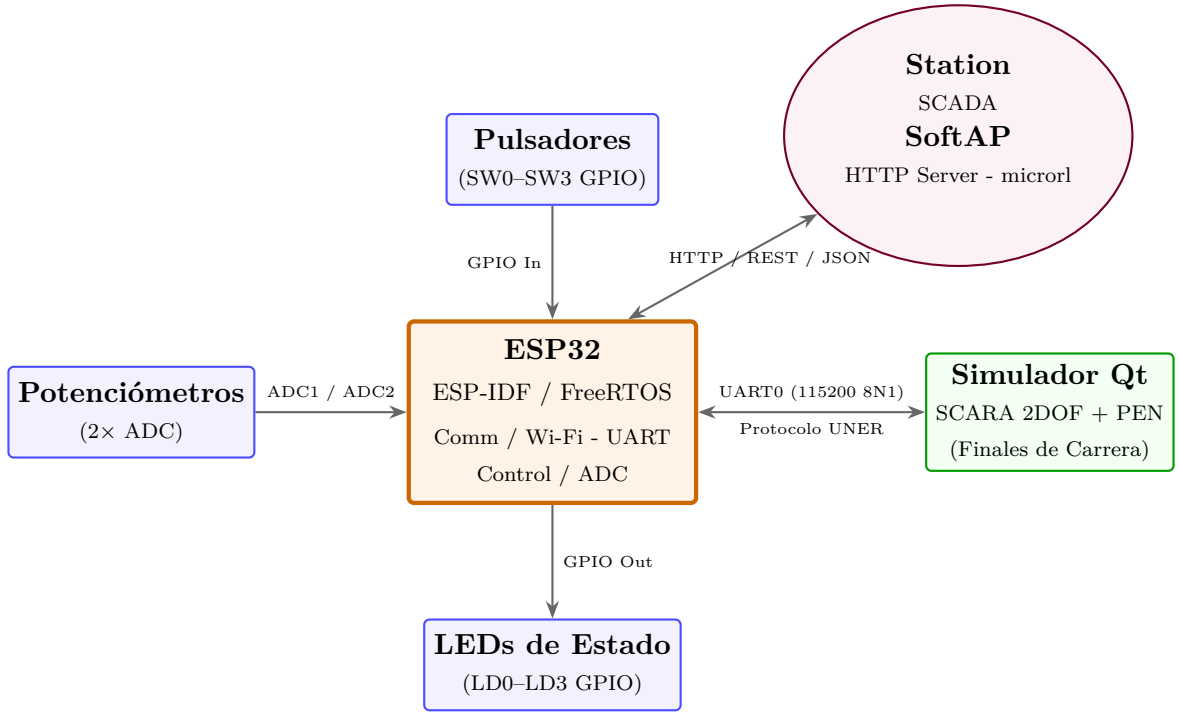


Figura 1: Diagrama de Contexto del Sistema SCARA 2DOF y Firmware ESP32

1.1. Hardware Disponible y Configuración

- **Unidad Central de Procesamiento (MCU):** ESP32 (Dual-Core Xtensa LX6, FreeRTOS Nativo / ESP-IDF).
- **Actuadores:**
  - 2 Motores DC de ∼50 RPM controlados por salidas PWM.
  - Actuador de elevación/descenso de lápiz (**PEN**) mediante servomotor o solenoide.
- **Sensores:**
  - 4 Finales de carrera de límite angular: `ARM1_L`, `ARM1_R`, `ARM2_L`, `ARM2_R`.
  - 1 Final de carrera para movimiento máximo hacia atrás: `LIMIT_SW`.

  - 2 Potenciómetros lineales acoplados a canales de conversión analógica-digital (**ADC**).
- **Interfaz de Usuario Local (Protoboard):**
  - 4 Pulsadores: `SW0`, `SW1`, `SW2`, `SW3`.
  - 4 LEDs indicadores: `LD0` (`ARM1_L`), `LD1` (`ARM1_R`), `LD2` (`ARM2_L`), `LD3` (`ARM2_R`).
- **Módulos de Comunicación:**
  - Puerto serie `UART0` configurado a 115200 baudios (8N1).
  - Wi-Fi Dual Mode: Punto de acceso (**SoftAP**) para configuración e interfaz web y modo Estación (**Station**) para telemetría.

2. Protocolo de Comunicación UART (Protocolo UNER)

El simulador Qt envía y recibe tramas binarias sobre el puerto `UART_NUM_0` a **115200 8N1**. El intercambio de datos se realiza mediante la especificación de comandos de la Tabla 1.

Cmd	Nombre	Origen → Destino	Datos Enviados	Respuesta ESP32 / Descripción
`0xF0`	**ALIVE**	ESP32 → Qt	Ninguno	`0xF0 0x0D` *Handshake de comprobación de conexión.*
`0xD0`	**GET STATE**	ESP32 → Qt	Ninguno	`(int32_t)angle1`, `(int32_t)angle2`, `(int32_t)pwr1`, `(int32_t)pwr2`, `(uint8_t)stateArm` *Solicitud de telemetría completa del estado.*
`0xD1`	**SET POWER**	ESP32 → Qt	`(int32_t)PWR1`, `(int32_t)PWR2`	`(int32_t)PWR1`, `(int32_t)PWR2` *Establece la potencia de ambos motores.*
`0xD2`	**SET PEN**	ESP32 → Qt	`(uint8_t)statePEN` (0:OFF, 1:ON)	`(uint8_t)statePEN` *Comanda la posición del lápiz.*
`0xD3`	**CLEAR**	ESP32 → Qt	Ninguno	`0xD3 0x0D` *Limpia el lienzo de dibujo en la interfaz Qt.*
`0xD4`	**LIMIT REACHED**	Qt → ESP32	`stateArm` (Bitmap)	Retorna `stateArm` ACK. *Enviado por el simulador Qt al activarse un límite. El ESP32 actualiza `LD0..LD3`.* bit[0]: Pen state. bit[1]: LIMIT_SW. bit[2]: ARM1_L. bit[3]: ARM1_R. bit[4]: ARM2_L. bit[5]: ARM2_R. bit[6..7]: dummy.

Tabla 1: Tabla de comandos del Protocolo Serie UNER.

3. Consignas y Requerimientos del Firmware en ESP-IDF

3.1. MÓDULO A: Arquitectura FreeRTOS y Concurrencia

Diseñe la arquitectura de software del sistema basándose en el framework ESP-IDF. Asigne las tareas a los núcleos del ESP32 (*Core Affinity*), seleccione prioridades y utilice mecanismos de sincronización (`Queues`, `Mutex`, `Event Groups`) de manera justificada.

1. **Lectura e Inserción ADC:**
   - Implementar la lectura continua u *oneshot* con la API de ESP-IDF para los dos potenciómetros lineales.
   - Aplicar un filtro digital (p. ej., promedio móvil) para reducir el ruido térmico y las no linealidades propias del ADC del ESP32.
2. **Gestión de LEDs y Finales de Carrera (CMD ID `0xD4`):**
   - El simulador Qt detecta las colisiones y envía el comando `0xD4 [stateArm]`.
   - La tarea receptora de UART debe interpretar este paquete y actualizar inmediatamente los niveles lógicos de los GPIOs asignados a los LEDs en la protoboard:
     - `LD0`: Representa el final de carrera `ARM1_L`.
     - `LD1`: Representa el final de carrera `ARM1_R`.
     - `LD2`: Representa el final de carrera `ARM2_L`.
     - `LD3`: Representa el final de carrera `ARM2_R`.
     - `LD0..LD3`: Representa el desplazamiento máximo hacia atrás `LIMIT_SW`. Cuando se da esta situación todos los leds deben parpadear 3 veces, encendiendosé durante 100ms y apagándose durante 100ms. Luego deben seguir manteniendo el estado anterior a que se de esta situación.
3. **Gestión de Tareas de Recepción y Transmisión:**
   - **Tarea de Recepción:** Debe implementarse una única tarea encargada de procesar los datos recibidos desde UART0, TCP Server y TCP Client, gestionados mediante un único buffer de recepción (Figura 2).
   - **Tarea de Transmisión:** Debe implementarse una única tarea encargada de enviar los datos hacia UART0, TCP Server y TCP Client, gestionados mediante un único buffer de transmisión.
   - **Mecanismo de Comunicación:** Se recomienda el uso de colas de mensajes (*Queues*) de FreeRTOS para la transferencia e interconexión de datos en ambas tareas (Figura 3).

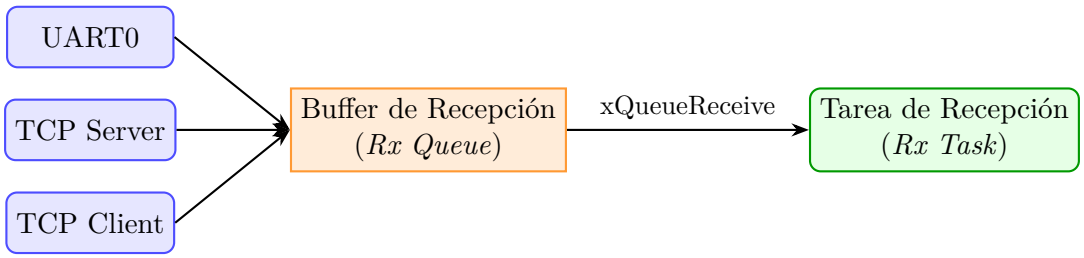


Figura 2: Arquitectura de la Tarea y Buffer de Recepción.

4. **Control del SCARA por Modos:**
   - **MODO A:**
     - **Manual Hardware:** El movimiento del SCARA es controlado localmente. La posición gruesa proviene de los potenciómetros y los pulsadores (`SW0` = +angle1 – `SW1` = -angle1; `SW2` = +angle2 – `SW3` = -angle2) permiten realizar un ajuste fino (0.5º) incremental sobre los ángulos de las articulaciones.
     - **Manual Comandos:** El movimiento del SCARA es controlado localmente mediante comandos enviadados al servidor TCP utilizando *microrl*.

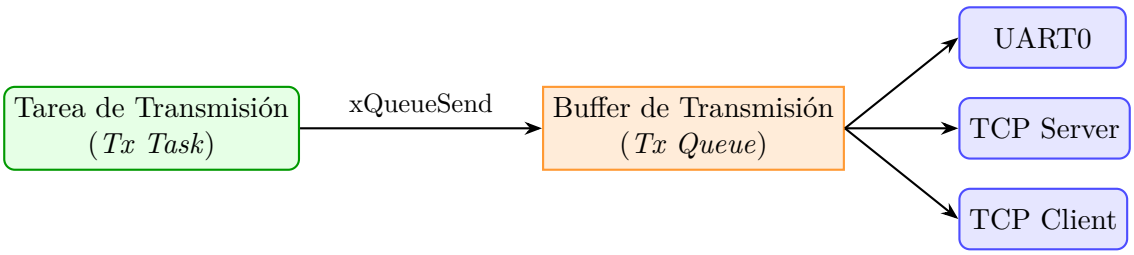


Figura 3: Arquitectura de la Tarea y Buffer de Transmisión.

- **MODO B (Autónomo):** Una tarea independiente de FreeRTOS ejecuta algoritmos de trayectorias preprogramadas para escribir **números de 3 dígitos** de manera autónoma. En este modo, se deshabilitan las lecturas del ADC y pulsadores locales para el control directo.

3.2. MÓDULO B: Protocolo de Comunicación UART

Implementar el driver UART de ESP-IDF (`driver/uart.h`) utilizando una máquina de estados para la recepción de tramas y el despacho de comandos del Protocolo UNER (Tabla 1) de manera no bloqueante.

3.3. MÓDULO C: Redes y Servidor Web (SoftAP + Station)

1. **Punto de Acceso (SoftAP):**
   - Configurar el módulo Wi-Fi del ESP32 en modo SoftAP.
   - Inicia un servidor TCP en el puerto 30000 que permita alojar una página web interactiva (HTTP Rest) y recibir comandos de texto utilizando *microrl* y PuTTY.
     - **HTTP Rest:** Panel de monitoreo de estado en tiempo real (Ángulos, Potencia, Estado de finales de carrera y estado del lápiz **PEN**).
     - **COMANDO por *microrl*:** El movimiento del SCARA es controlado localmente por medio de una teminal de comandos (Tabla 2) (implementada en softAP) utilizando `microrl`. Considere agregar los comandos que crea de su utilidad.

COMANDO	PARÁMETROS	RESPUESTA
STATE Devuelve el estado del brazo	Sin parámetros	pwr1, pwr2 angle1, angle2 state
MOVE Mueve el brazo hasta el ángulo indicado	pwr1 angle1 pwr2 angle2	angle1 angle2
NUMERO Escribe el número (MODO B)	número	número
SETMODE Selecciona el modo de funcionamiento (A, B) Sin parámetros devuelve el modo actual.	mode [A, B]	mode
WIFI Wifi Local Sin parámetros devuelve el estado y SSID	SSID PASSWORD	ssid password
SCADA Configuración SCADA Sin parámetros devuelve estado y valores de IP:PORT	IP PORT	ip port
STOP Detiene los Moviemientos		

Tabla 2: Tabla de comandos `microrl`.

2. **Modo Estación (Station):**
   - Al recibir y validar las credenciales a través de la interfaz microrl, el ESP32 debe conectarse a la red local en modo Station.
   - Conectado en modo Station al recibir y validar las credenciales del servidor SCADA conectarse al mismo.
   - Transmitir de forma no bloqueante la telemetría (ADCs, PWRs, Finales de carrera, Modo de funcionamiento) del robot hacia un servidor HTTP Rest centralizado, sin perturbar el tiempo real del control de movimiento.
   - ENDPOINT - JSON para el Servidor:

```
SERVER: ip:port

POST /scara/telemetry HTTP/1.1\r\n
Content-Type: application/json\r\n
Content-Length: nBytes\r\n\r\n  (nBytes: bytes del body)
{
  "NAME": "MyName",
  "ADC1": valueADC1,
  "ADC2": valueADC2,
  "PWR1": pwr1,
  "PWR2": pwr2,
  "MODO": "modo",  (caracter "A" o "B")
  "STATE": state   (int representado en el estado del brazo)
}

Respuesta Servidor:

HTTP/1.1 200 OK\r\n
Content-Type: application/json\r\n
Content-Length: 13\r\n\r\n  o Content-Length: 16\r\n\r\n
{"ACK": "OK"}                 {"ACK": "ERROR"}
```